本节课导学提纲

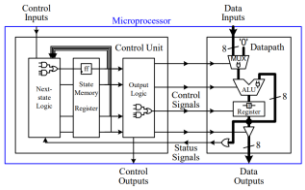
本节知识点：

- 课程概况
- 进位计数制：十进制数及任意进制数表示方法
- 常用进制数：BIN、OCT、HEX
- 数制之间的相互转换
- 编码：BCD码、Gray码
- 有符号数、运算及溢出

主 讲：任爱锋
教 师：袁晓光、何学辉、陶海红、
侯彦宾、董玫、杨旭、宗汝

课程概况

数字逻辑与微处理器：融合数字逻辑与系统设计（数字电路）和微机原理与接口设计（微机原理）



微处理器内部结构：

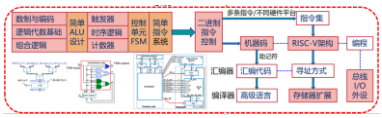
- 数据通路（Datapath）
  - ALU-完成数据运算
  - 寄存器（Register）-数据的临时缓存
  - 数据选择器（MUX）
  - 逻辑门
  - 数据信号线（包括单线、总线Bus）等
- 控制单元（Control Unit）
  - 控制数据通路的所有操作
  - 包括Next-state Logic、State Memory和Output Logic

数字逻辑是构成微处理器的基础！

《数字逻辑与微处理器》课程学时设置：

数字逻辑与微处理器

4学分
48+16课时
(线下线上混合)



课程概况

数字逻辑与微处理器课程线上线下混合式教学课程安排

数字逻辑与微处理器技术基础	数字理论基础	数制与编码	（4学分）48学时+16线上MOOC	
		逻辑代数基础		
	微处理器结构与数字逻辑	微处理器基本结构与功能		
		组合逻辑电路与运算逻辑单元（ALU）		
		触发器（DFF）结构及功能		
		时序逻辑电路		
	总线与接口外设	集成逻辑门及总线驱动		
		总线及其形成		
		存储器及可编程逻辑器件		
	指令结构及程序设计	CPU设计及指令执行方式		
		CPU指令系统		
		机器语言编码设计		
配套实验	微处理器程序设计实验	微处理器编程（理论）	16学时	
		程序设计（实验）		
	数字电路与EDA实验	硬件描述语言（理论）	16学时	
		综合设计（实验）		

线下理论48学时，实验32学时；线上慕课16学时。

在线课程内容

配套慕课	进制及转换	16学时
	代数法化简	
	TFF、JKFF	
	脉冲波形产生与整形	
	8259A/8253/8255A	
	ADC/DAC	

考核方式：

（1）平时成绩：30%。主要考核对每堂课知识点的复习、理解和掌握程度；作业及综合设计的研讨、交流、互动等情况，以及线上资源学习情况。

（2）期中考试成绩：10%。主要考核数字逻辑基础部分的基本概念、分析方法和设计方法的掌握程度。

（3）期末考试成绩：60%。考核课程全部内容，包括基本概念、电路的分析/设计方法、程序的分析/设计方法、较复杂电路设计及控制等。

希望同学上课积极配合、利用好线上资源及各种信息化工具

一. 数制.

$(N)_R = a_{n-1} a_{n-2} \cdots a_1 a_0 a_{-1} \cdots a_{-m}$

$= a_{n-1} \times R^{n-1} + \cdots + a_0 \times R^0 + a_{-1} \times R^{-1} + \cdots + a_{-m} \times R^{-m}$

$= \sum_{i=-m}^{n-1} a_i \times R^i$

$a_i \in [0, R-1)$

(1) $R=10$. a_i : 0~9. Dec 十进制

(2) $R=8$ a_i : 0~7. Oct

(3) $R=16$ a_i : 0~15 Hex

(4) $R=2$ a_i : 0~1. Bin

$R \Longleftrightarrow$ Dec. $\begin{cases} \text{整数} / R \text{ 取余} \\ \text{小数} \times R \text{ 取整} \end{cases}$

Bin $\Longleftrightarrow \begin{cases} \text{Oct} \\ \text{Hex.} \end{cases}$

$\longleftarrow \cdot \longrightarrow$
小数

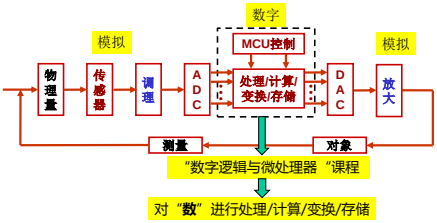
Bin四位整合换算成 Oct.

例如. 0110 Bin $\rightarrow 1\times8^2+1\times8^1+0\times8^0 \rightarrow$

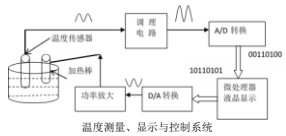
二. 编码：

(1) PCB码

一个通用硬件系统基本结构：课程关系



实际应用系统？



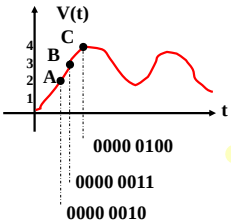
温度测量、显示与控制系统

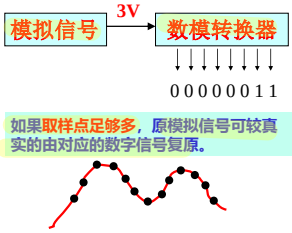
第1章 数制与编码

模拟 随时间的连续变化其值也连续变化，如声音、电压、电流、压力、速度、温度等

数字 时间上和数值上是离散的，数值的大小和每次的增减都是某一个单位（称为量化单位）的整数倍

模拟量用数字0、1的编码表示（模拟量的数字化）：





如果取样点足够多，原模拟信号可较真实的由对应的数字信号复原。

将模拟信号转换为数字信号的优点？

第1章 数制与编码

数制 如：十进制数 5555（五千五百五十五）

进位计数制中需要理解的三个术语：

（1）进位制： 进位计数制，简称进位制。

（2）基数（底数）：进位制用到的数码个数，即“逢几进一”中的“几”。

（3）位权（位的权数）：在某一进位制中，每一位的大小都对应着该位上的数码乘上一个固定的数，这个固定的数就是这一位的权数。

十进制数表示方法：

$$435.86 = 4\times10^2 + 4\times10^1 + 5\times10^0 + 8\times10^{-1} + 6\times10^{-2}$$

位置记数法或并列表示法　　多项式表示法或按权展开法

$4\times10^2 + 4\times10^1$

任何一个十进制数N，用一个通式如何表示？

任何一个十进制数N，都可以用位置记数法和多项式表示法写为：

$$(N)_{10} = a_{n-1}a_{n-2}\cdots a_1a_0 \cdot a_{-1}a_{-2}\cdots a_{-m}$$
$$= a_{n-1}\times 10^{n-1} + a_{n-2}\times 10^{n-2} + \cdots + a_1\times 10^1 + a_0\times 10^0 + a_{-1}\times 10^{-1}$$
$$+ a_{-2}\times 10^{-2} + \cdots + a_{-m}\times 10^{-m}$$
$$= \sum_{i=-m}^{n-1} a_i \times 10^i$$

- 式中，n代表整数位数，m代表小数位数
- a_i（$-m \le i \le n-1$）第i位数码，取值0,1,…,9
- 10^i为第i位数码的权值

推广到任意进制数： 对于一个基数为$R(R\ge 2)$的R进制数N可以写为

$$(N)_R = a_{n-1}a_{n-2}\cdots a_1a_0 \cdot a_{-1}a_{-2}\cdots a_{-m} \qquad 0 \le a_i \le R-1$$
$$= a_{n-1}\times R^{n-1} + a_{n-2}\times R^{n-2} + \cdots + a_1\times R^1 + a_0\times R^0 + a_{-1}\times R^{-1}$$
$$+ a_{-2}\times R^{-2} + \cdots + a_{-m}\times R^{-m}$$
$$= \sum_{i=-m}^{n-1} a_i R^i$$

- 式中，n代表整数位数，m代表小数位数
- a_i（$-m \le i \le n-1$）第i位数码，取值0,1,…,(R-1)
- R^i为第i位数码的权值

7

$$(N)_R = a_{n-1}a_{n-2}\cdots a_1a_0 \cdot a_{-1}a_{-2}\cdots a_{-m} \qquad 0 \le a_i \le R-1$$
$$= a_{n-1}\times R^{n-1} + a_{n-2}\times R^{n-2} + \cdots + a_1\times R^1 + a_0\times R^0 + a_{-1}\times R^{-1}$$
$$+ a_{-2}\times R^{-2} + \cdots + a_{-m}\times R^{-m}$$

思考：

$$= \sum_{i=-m}^{n-1} a_i R^i$$

基数R、数码a_i、位权R^i

- 二进制数(Bin)：数码0、1；基数2=2¹。运算规律：逢2进1，即1+1=10。

如：$(101.01)_2 = 1\times 2^2 + 0\times 2^1 + 1\times 2^0 + 0\times 2^{-1} + 1\times 2^{-2} = (5.25)_{10}$

Bin

运算规则	
加法规则：	0+0=0，0+1=1，1+0=1，1+1=10
乘法规则：	0·0=0，0•1=0，1•0=0，1•1=1

为什么引入八进制、十六进制？

- 八进制数(Oct)：数码0～7；基数8=2³。运算规律：逢8进1，即7+1=10。

如：$(207.04)_8 = 2\times 8^2 + 0\times 8^1 + 7\times 8^0 + 0\times 8^{-1} + 4\times 8^{-2} = (135.0625)_{10}$

- 十六进制数(Hex)：数码0～9、A～F；基数16=2⁴。运算规律：逢16进1，即F+1=10。

如：$(D8.A)_{16} = 13\times 16^1 + 8\times 16^0 + 10\times 16^{-1} = (216.625)_{10}$

任意进制数：例如五进制数？$(243.12)_5$=?

8

$$(N)_R = a_{n-1}a_{n-2}\cdots a_1a_0 \cdot a_{-1}a_{-2}\cdots a_{-m} \qquad 0 \le a_i \le R-1$$
$$= a_{n-1}\times R^{n-1} + a_{n-2}\times R^{n-2} + \cdots + a_1\times R^1 + a_0\times R^0 + a_{-1}\times R^{-1}$$
$$+ a_{-2}\times R^{-2} + \cdots + a_{-m}\times R^{-m}$$

思考：

- 为什么要引入Bin、Oct和Hex?
- 不同进制之间是如何相互转换的?
  - ✓ 任意进制数转换为十进制数!
  - ✓ 十进制数转换为任意进制数?

C语言编程打印任意输入的十进制数的整数各位的数码

```
#include <stdio.h>
int main() {
    int num;    // 存放输入的十进制数
    printf("请输入一个十进制数：");
    scanf("%d", &num);
    while (num != 0) {
        int digit = num % 10;  // 取得最低位的数字
        printf("%d ", digit);  // 打印该数字
        num /= 10;             // 去除已经处理过的最低位
    }
```

C语言编程打印任意输入的十进制数的小数部分的数码

```
#include <stdio.h>
int main() {
    double num, fractionalPart;
    int digit;
    printf("请输入一个十进制数：");
    scanf("%lf", &num);
    // 提取小数部分
    fractionalPart = num - (int)num;
    // 循环打印小数部分的每一位
    while (fractionalPart > 0) {
        // 乘以10后取整得到当前小数位
        digit = (int)(fractionalPart * 10);
        printf("%d ", digit);
        // 更新小数部分为剩余部分
        fractionalPart = fractionalPart * 10 - digit;
    }
    printf("\n"); // 输出换行
    return 0;
}
```

如果把这里的基数10改为2、8、16等，会得到什么结果？

9

乘R没整数.

总结：十进制数转换为其它进制数

将一个带有整数和小数的十进制数转换成任意进制R数时，必须将整数部分按除R取余法，小数部分按乘R取整法进行转换，然后再将两者的转换结果合并起来。如$(105.4)_{10}=(\ ?\ )_2$

$(105)_{10}=(\ ?\ )_2$

		余数
2	105	1 a_0
2	52	0 a_1
2	26	0 a_2
2	13	1 a_3
2	6	0 a_4
2	3	1 a_5
2	1	1 a_6
	0	

$(105)_{10}=(1101001)_2$

小数部分乘2取整的过程，不一定能使最后乘积为0，因此转换值存在误差。

$(0.4)_{10}=(\ ?\ )_2$

	整数
0.4 × 2 = 0.8	0 a_{-1}
0.8 × 2 = 1.6	1 a_{-2}
0.6 × 2 = 1.2	1 a_{-3}

$(0.4)_{10}\approx(0.011)_2$

故：$(105.4)_{10}\approx(1101001.011)_2$

10

任意R进制数 —多项式展开计算→ 十进制数 —整数除R取余 小数乘R取整→ P进制数

思考：二进制数与八进制数、十六进制数之间的相互转换？

R=2: 二进制数(Binary) 2^1
R=8: 八进制数(Octal) 2^3
R=16: 十六进制数(Hexadecimal) 2^4

感觉：3位二进制数恰好相当1位八进制数，4位二进制数相当1位十六进制数

例如：把一个二进制数N转换为8进制数：

$(N)_2=(b_7b_6b_5b_4b_3b_2b_1b_0b_{-1}b_{-2}b_{-3}b_{-4}b_{-5})_2=(\ ?\ )_8$

$(N)_2=b_7 2^7+b_6 2^6+b_5 2^5+b_4 2^4+b_3 2^3+b_2 2^2+b_1 2^1+b_0 2^0+b_{-1}2^{-1}+b_{-2}2^{-2}+b_{-3}2^{-3}+b_{-4}2^{-4}+b_{-5}2^{-5}$

$=(0\times 2^8+b_7 2^7+b_6 2^6)+(b_5 2^5+b_4 2^4+b_3 2^3)+(b_2 2^2+b_1 2^1+b_0 2^0)+(b_{-1}2^{-1}+b_{-2}2^{-2}+b_{-3}2^{-3})+(b_{-4}2^{-4}+b_{-5}2^{-5}+0\times 2^{-6})$

$=2^6(0\cdot 2^2+b_7 2^1+b_6 2^0)+2^3(b_5 2^2+b_4 2^1+b_3 2^0)+2^0(b_2 2^2+b_1 2^1+b_0 2^0)+2^{-3}(b_{-1}2^2+b_{-2}2^1+b_{-3}2^0)+2^{-6}(b_{-4}2^2+b_{-5}2^1+0\cdot 2^0)$

$=8^2(0\cdot 2^2+b_7 2^1+b_6 2^0)+8^1(b_5 2^2+b_4 2^1+b_3 2^0)+8^0(b_2 2^2+b_1 2^1+b_0 2^0)+8^{-1}(b_{-1}2^2+b_{-2}2^1+b_{-3}2^0)+8^{-2}(b_{-4}2^2+b_{-5}2^1+0\cdot 2^0)$

$=8^2 o_2+8^1 o_1+8^0 o_0+8^{-1}o_{-1}+8^{-2}o_{-2}$

$=(o_2 o_1 o_0 o_{-1} o_{-2})_8$

Bin to Oct
Oct to Bin

把二进制数转换为十六进制数？

11

数制与编码

总结：二进制数与八进制数、十六进制数之间的相互转换

1）二进制数转换为八进制、十六进制数

三（四）位一组，不足左补零 ← 小数点 → 三（四）位一组，不足右补零

2）八进制、十六进制数转换为二进制数

八进制数 ⟶ 二进制数：1位八进制数码变3位二进制数

十六进制数 ⟶ 二进制数：1位十六进制数码变4位二进制数

3）任意进制数之间的相互转换

R进制数 —多项式展开计算→ 十进制数 —整数除R取余 小数乘R取整→ P进制数

最熟悉的进制数

12

例 1: $(110101.011000111)_2 = (\quad ? \quad)_8 = (\quad ? \quad)_{16}$

110	101	.	011	000	111				
6	5	.	3	0	7				
0011	0101	.	0110	0011	1000				
3	5	.	6	3	8				

$(110101.011000111)_2 = (65.307)_8 = (35.638)_{16}$

例 2: $(2EA)_{16} = (\quad ? \quad)_{10}$

2	E	A
0010	1110	1010

$(001011101010)_2 = 512+128+64+32+10=746_{10}$

$(2EA)_{16} = (746)_{10}$

用一定位数的二进制数来表示十进制数码、字母、符号等信息称为编码。

这一定位数的二进制数就称为代码。

对于N个信息，要用几位二进制数才能满足编码呢？

$$2^n \geq N$$

重点掌握：二—十进制编码(BCD码)

用4位二进制数的10种组合表示十进制数的0~9，简称BCD码(Binary Coded Decimal)。

$$A_{16}^{10} = \frac{16!}{(16-10)!} \approx 2.9 \times 10^{10}$$

二—十进制编码(BCD码)

- 8421 BCD 码： **有权码，且编码唯一！**

$(a_3a_2a_1a_0)_{8421BCD}=8\times a_3+4\times a_2+2\times a_1+1\times a_0$

- 5421 BCD 码： **有权码，且编码不唯一！**

$(a_3a_2a_1a_0)_{5421BCD}=5\times a_3+4\times a_2+2\times a_1+1\times a_0$

- 2421 BCD 码： **有权码，且编码不唯一！**

$(a_3a_2a_1a_0)_{2421BCD}=2\times a_3+4\times a_2+2\times a_1+1\times a_0$

- 余3码 码： **无权码，且编码唯一！**

$(b_3b_2b_1b_0)_{余3码}=(a_3a_2a_1a_0)_{8421BCD}+(0011)_2$

0 0 0 0
0 0 0 1
0 0 1 0
0 0 1 1
0 1 0 0
0 1 0 1
0 1 1 0
0 1 1 1
1 0 0 0
1 0 0 1
1 0 1 0
1 0 1 1
1 1 0 0
1 1 0 1
1 1 1 0
1 1 1 1

表 1-2 几种常用的BCD码

Decimal	8421 BCD	5421 BCD	2421 BCD	余3码
0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 0 0 1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 0 1 0	0 0 1 0	0 1 0 1
3	0 0 1 1	0 0 1 1	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 0 0	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0	1 0 1 1	1 0 0 0
6	0 1 1 0	1 0 0 1	1 1 0 0	1 0 0 1
7	0 1 1 1	1 0 1 0	1 1 0 1	1 0 1 0
8	1 0 0 0	1 0 1 1	1 1 1 0	1 0 1 1
9	1 0 0 1	1 1 0 0	1 1 1 1	1 1 0 0
unused	1 0 1 0	0 1 0 1	0 1 0 1	0 0 0 0
	1 0 1 1	0 1 1 0	0 1 1 0	0 0 0 1
	1 1 0 0	0 1 1 1	0 1 1 1	0 0 1 0
	1 1 0 1	1 1 0 1	1 0 0 0	1 1 0 1
	1 1 1 0	1 1 1 0	1 0 0 1	1 1 1 0
	1 1 1 1	1 1 1 1	1 0 1 0	1 1 1 1

特点：对9的自补码

任何一个十进制数均可以用BCD码表示，如：

$$432.25=(0100\ 0011\ 0010.0010\ 0101)_{8421BCD}$$
$$=(0111\ 0110\ 0101.0101\ 1000)_{余3}$$
$$=(\qquad ? \qquad)_2$$

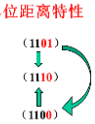
可靠性编码

1. Gray码(格雷码)

Gray码也称循环码，其最基本的特性是任何相邻的两组代码中，仅有一位数码不同，因而又叫单位距离码。

Decimal	$B_3\ B_2\ B_1\ B_0$	$G_3\ G_2\ G_1\ G_0$
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1
10	1 0 1 0	1 1 1 1
11	1 0 1 1	1 1 1 0
12	1 1 0 0	1 0 1 0
13	1 1 0 1	1 0 1 1
14	1 1 1 0	1 0 0 1
15	1 1 1 1	1 0 0 0

单位距离特性



n

典 型 的 格 雷 码 写 法

两位格雷码	三位格雷码	四位格雷码
0 0	0 0 0	0 0 0 0
0 1	0 0 1	0 0 0 1
1 1	0 1 1	0 0 1 1
1 0	0 1 0	0 0 1 0
	1 1 0	0 1 1 0
	1 1 1	0 1 1 1
	1 0 1	0 1 0 1
	1 0 0	0 1 0 0
		1 1 0 0
		1 1 0 1
		1 1 1 1
		1 1 1 0
		1 0 1 0
		1 0 1 1
		1 0 0 1
		1 0 0 0

低位镜像，高位取反

2. 奇偶校验码 (Odd/Even codes）

代码(或数据)在传输和处理过程中，有时会出现代码中的某一位由 0 错变成 1，或 1 变成 0。奇偶校验码是一种具有检验出这种错误的代码，奇偶校验码由信息位和一位奇偶检验位两部分组成。

信息位：是位数不限的任一种二进制代码。

检验位：仅有一位，它可以放在信息位的前面，也可以放在信息位的后面。

2、奇偶校验码 (Odd/Even codes）

编码方法：

信息位	检验位

使得一组代码中信息位和检验位中“1”的个数之和为奇数，称为奇检验（odd codes）；

使得一组代码中信息位和检验位中“1”的个数之和为偶数，称为偶检验（Even codes）。

表 1-4 带奇偶检验的8421 BCD码

十进制数	8421 BCD 奇检验		8421 BCD 偶检验	
0	0000	1	0000	0
1	0001	0	0001	1
2	0010	0	0010	1
3	0011	1	0011	0
4	0100	0	0100	1
5	0101	1	0101	0
6	0110	1	0110	0
7	0111	0	0111	1
8	1000	0	1000	1
9	1001	1	1001	0
	信息位	检验位	信息位	检验位

有符号数的表示

为了表示一个有符号数，除了数值位以外，还应制定符号位，通常以这个数的最高位表示符号位。

我们假定讨论的数为整数，对8位有符号二进制整数表示为：

$D_7D_6D_5D_4D_3D_2D_1D_0$

数值部分

0：表示正数
1：表示负数
符号位

0 1

1. 原码表示法

约定数码序列中的最高位为符号位；
正数的符号位用0表示，负数的符号位用1表示；
其余有效数值部分则用二进制的绝对值表示。

一个数X的原码记为：$[X]_{原}$

$D_7D_6D_5D_4D_3D_2D_1D_0$

0	0	0	0	0	0	1	1

+3的表示 $[+3]_{原}$ =00000011B

1	0	0	0	0	0	1	1

-3的表示 $[-3]_{原}$ =10000011B

原码表示法的优点是直观易懂，但在进行算术运算时会带来麻烦！

数0的两种表示方法

$[+0]_{原}$ =0 0 0 0 0 0 0 0B　　$[-0]_{原}$ =1 0 0 0 0 0 0 0B

对8位有符号二进制数用原码表示的范围：
正数从00000000～01111111，+0～+127
负数从10000000～11111111，-0～-127

2. 补码表示法（类似于时钟倒拨3小时，与正拨9小时效果相同！）

一个数X的补码记为 $[X]_{补}$ ，补码定义为：

$$[x]_{补}=\begin{cases} x & 0 \le x < 2^{n-1} \\ 2^n + x \quad 当 & -2^{n-1} \le x < 0 \end{cases} \pmod{2^n}$$

正数的补码=原码，即 $[x]_{补}=[x]_{原}$ ，只有负数求补的问题

补码的算法：负数的补码=2^n+负数

以8位有符号数为例： -1的补码 = 2^8 -1 = 255 = 11111111

-2的补码 = 2^8 -2 = 254 = 11111110

值的注意的是：0的补码只有唯一的形式，符号位和数值位均为0。

用8位补码计算 7-2= $(00000111)_2+(11111110)_2=(00000101)_2=5$

借助于补码，负数就没有了，从而把“减法转换为加法运算”

25

2. 补码表示法——求负数的补码一般方法

正数：$[x]_{补}=[x]_{原}$

负数：除符号位外，其它位取反再加1

$[[X]_{补}]_{补} = [X]_{原}$

- 计算机中所有的有符号数字都是补码表示
- 某8位内存单元存储的有符号数是D2H，则其所表示的十进制数是多少？

$[X]_{补}$ = D2H=1 1010010B

X = $[[X]_{补}]_{补}$ = $[11010010]_{补}$

= - 0101110B= - 2EH

所以：X = - 46

26

数制与编码

总结

课程概况

进位计数制：十进制数及任意进制数表示方法

常用进制数：BIN、OCT、HEX

数制之间的相互转换

编码：BCD码、Gray码

有符号数、运算及溢出

27